Is `diffThreshold` Just an Interior Certificate? The All-MAXITER Split Rule

Matias Saavedra

Wednesday 3rd June, 2026

Branch: examine/maxiter-split atop binary-v5-affinity (fc33e29); *not* merged (a characterization). Machine: i7-9750H + GTX 1660 Ti Max-Q. Data: experiments/18-maxiter-split/.

Resumen

The subdivision heuristic computes a region (rather than splitting it) when its nine stencil samples are "uniform": `maxIter`−`minIter` < `diffThresh`·`maxIter`. Inspecting the visualisations suggests that at the tuned `diffThresh` = 0.1 this is, in practice, just "don't split if the region is interior" — the cheapest way for nine samples to be similar is for all of them to run to `MAXITER`. This report tests that by replacing the relative-spread test with a parameter-free binary rule: compute iff all nine samples reached `MAXITER` (`minIter`=frame `MAXITER`), else split. The result is decisive: the two rules produce **byte-identical** decompositions on the production zoom (**5608** leaves both) and on three of the four report-13 regimes. They diverge only on **outside** (uniform deep exterior): there `diffThreshold` keeps a large uniform-exterior region as one leaf, while the binary rule splits it to the floor (+22 % leaves) — at *zero* wall cost, since those leaves are trivial iter-2 exterior. So `diffThreshold` = 0.1 *is* a parameter-free interior certificate on realistic content; the threshold's only distinctive job is to avoid shattering large uniform-exterior regions, which is cost-free here. This explains why 0.1 is the sweet spot.

1. Setup

The decision under test.

`MandelRegion::examine` computes a region or splits it into four by one line:

Código 1: mandelregion.cpp – the split decision (current vs the experiment).

```
// current (diffThreshold relative-spread test):
if (maxIter - minIter < diffThresh * maxIter || pixelsX * pixelsY < pixelSizeThresh)
// experiment (binary "all 9 samples in-set"):
if (minIter == ownerFrame->MAXITER      || pixelsX * pixelsY < pixelSizeThresh)
```

The hypothesis (from reading the `viz` overlays, reports 08/13): at `diffThresh` = 0.1 the relative test is so tight that exterior regions almost always split, and the only regions uniform enough to be computed are the interior ones, where all nine samples reach `MAXITER`. If so, the parameter-free rule on the right — compute iff

every sample is in-set — should reproduce the same decomposition. The pixel floor (`pixelSizeThresh`) is unchanged, so boundary regions still terminate; `diffThresh` becomes unused.

Method.

Leaf counts are deterministic — the decomposition is thread-independent (verified byte-identical across executors in report 08) — so a single GPU-only run (`numThreads`=1) gives the exact leaf count per spec. Compared on the production `spec.in` (the canonical minibrot zoom) and the four report-13 zoom-point specs (`experiments/17-ncu-zoom-points/specs/`), all at `diffT` = 0.1, `pixT` = 32768. Wall time is measured GPU-only on the one spec where the rules differ.

2. Results

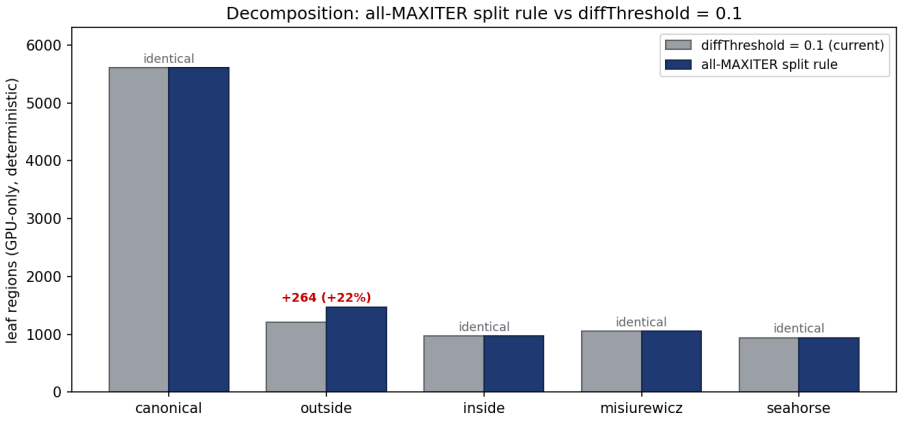

Figura 1: Leaf-region counts under the two rules. On the production zoom (`canonical`) and on inside / Misiurewicz / seahorse the two rules are *identical*; only `outside` differs (the binary rule splits large uniform-exterior regions the relative test keeps whole).

Tabla 1: Leaf counts (GPU-only, deterministic) and the wall time on the one spec where the rules differ. "identical" means byte-identical decomposition — same leaves, hence same compute and same wall.

spec	diffT=0.1	all-MAXITER	Δ
canonical `spec.in`	5,608	5,608	**identical**
inside	969	969	identical
misiurewicz	1,050	1,050	identical
seahorse	942	942	identical
outside	1,206	**1,470**	+264 (+22 %)
outside wall (GPU-only)	∼5.95 s	5.76–5.98 s	flat

3. Analysis

3.1. On realistic content, diffThreshold=0.1 *is* the interior certificate

Table 1, Figure 1: the binary all-MAXITER rule reproduces the diffT = 0.1 decomposition **exactly** on the production zoom (5608 leaves either way) and on inside, Misiurewicz, and seahorse. The hypothesis is confirmed: on this content the two rules are the same function.

The mechanism is that the relative-spread test *collapses* to the MAXITER test at diffThresh = 0.1. A region is computed only if maxIter − minIter < 0.1 · maxIter, i.e. all nine samples lie within 10 % of the largest. On boundary or interior content, the escape iteration varies sharply across the fractal boundary, so any region that is not pure interior has spread well above 10 % and splits; the only regions tight enough to compute are those where all nine samples reach MAXITER (spread exactly zero). The two predicates therefore select the same regions. This is why the canonical minibrot zoom — boundary- and interior-dominated — is byte-identical.

3.2. The one divergence: large uniform exterior

The rules differ only on outside: a zoom into $c = (1, 1)$, far from the set, where every pixel escapes at iteration 2. A large region there has all nine samples equal to 2 — spread zero, but *not* MAXITER. The relative test computes it as one big leaf; the binary rule, seeing samples that are not in-set, splits it to the pixel floor (+264 leaves, +22 %).

This is the one thing diffThreshold's relative formulation buys over a binary interior test: it recognises *any* uniform region, interior or exterior, and computes it whole. The implication, though, is that the difference is free: Table 1 shows the outside wall is flat (∼5.8 s under both), because the extra leaves are trivial iter-2 exterior — splitting them adds kernel launches but essentially no compute (work is conserved; reports 09/16). So the binary rule is not worse here in any way that costs time; it is only *less elegant*, shattering regions a relative test would keep whole.

3.3. Why diffThreshold=0.1 is the sweet spot

This reframes the diffThreshold tuning of reports 06/07/10. The reason 0.1 is optimal is that it is the threshold at which the rule becomes a clean interior certificate on boundary content: tight enough that only the genuinely in-set regions compute, loose enough not to fight the pixel floor. A larger diffThreshold (0.5) admits near-uniform but non-interior regions as leaves — computing regions that should have split — which is exactly the +1–2 % wall penalty reports 05/06 measured at 0.5. A smaller one (0.05) saturates against pixelSizeThresh and stops changing the decomposition. So 0.1 is not a delicate optimum; it is the operating point where the heuristic coincides with "compute the interior, split everything else," which the leaf counts here show is the decomposition the workload wants.

4. Implications

1. **The heuristic could be made parameter-free with no loss on this workload.** Replacing the diffThreshold test with the binary all-MAXITER rule removes a tuning knob and reproduces the production decomposition exactly

(5608 leaves). It is a legitimate simplification for Mandelbrot zooms dominated by boundary and interior content.

2. **But keep `diffThreshold` for robustness.** The binary rule degrades on uniform-*exterior* content (`outside`: +22 % leaves), shattering large coherent regions the relative test computes whole. The penalty is cost-free *here* (trivial exterior), but on a workload with expensive uniform-exterior regions it would add real launch and bookkeeping overhead. The relative test is the safer general rule; this experiment simply shows it is doing nothing extra on the canonical zoom.

3. **This is not a performance lever.** On the production workload the decompositions are identical, so wall time is identical. The value is explanatory: it pins down what `diffThreshold` = 0.1 actually does and why it is optimal — not a new optimisation. The branch is kept off `main` as a characterization (like the priority-queue branch, report 11).

5. Conclusion

Replacing the `diffThreshold` relative-spread test with a parameter-free "compute iff all nine samples reached `MAXITER`" rule reproduces the `diffT` = 0.1 decomposition **byte-identically** on the production zoom (5608 leaves) and on inside, Misiurewicz, and seahorse. The two rules diverge only on uniform deep exterior (`outside`, +22 % leaves), at zero wall cost. So on realistic Mandelbrot content `diffThreshold` = 0.1 is, exactly, a parameter-free interior certificate; the threshold's relative formulation only earns its keep on large uniform-exterior regions, which is cost-free here. This explains why 0.1 is the tuned optimum — it is the operating point where the heuristic coincides with "compute the interior, split everything else."